



Python程序设计

一、基础知识

教学内容：

Python语言概述和安装

Python基础知识：基本的输入输出

Python基础知识：变量和对象

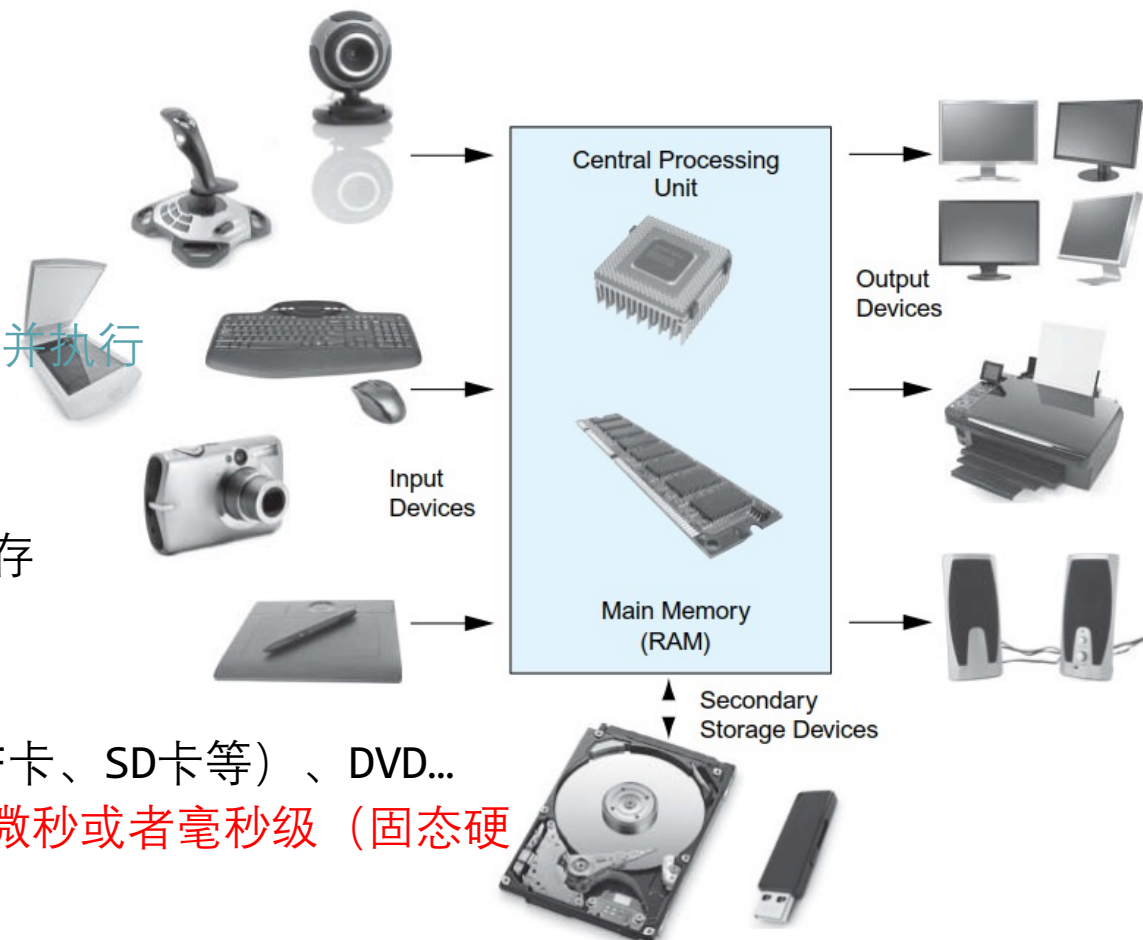
复旦大学版权所有

1.0 预备知识

现代计算机的部件：**硬件**和**软件**

硬件(hardware)：组成计算机的各个物理部件

- **CPU**(Central Processing Unit)：从内存中读取指令并执行
 - 有限的指令集
 - 按照顺序执行指令
 - 纳秒级的执行速度
- 主存/**内存**：RAM (Random Access Memory) 随机访问内存
 - 由许多电路组成
 - 易失性(Volatility):断电丢失
 - 访问速度快,纳秒级
- 辅存/**外存**(Secondary Storage)：硬盘、闪存 (U盘、TF卡、SD卡等)、DVD...
 - 持久性：断电后数据仍然存在 访问速度比内存慢, 微秒或者毫秒级 (固态硬盘速度提升只是十倍数量级, 但仍远小于内存)
 - 数据和指令首先从外存读到内存后才能被CPU使用
- **外设** (Peripheral)
 - 输入设备：键盘、鼠标、扫描仪、摄像头、麦克风等
 - 输出设备：显示器、打印机等
- **软件(software)**：待执行的程序、数据和文档
 - 程序(program)：一系列指令, 计算机根据指令进行运算完成相应的任务



数据(信息) 表达：Bit、Byte(K/M/G)

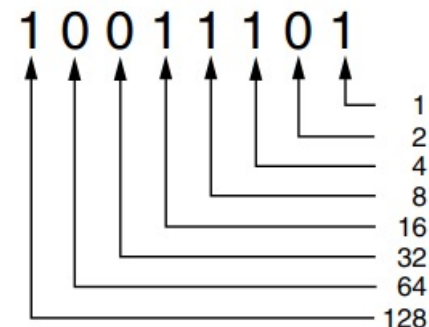
- 计算机的运算采用二进制

每个电路保存一个二进制数据(0或1)，称为位/比特 (bit)

连续8个比特组合在一起称为**字节(byte)**

存储单元的基本单位为**字节(byte)**，通过一个地址标识

二进制的10011101，对应十进制的 $1*2^7+1*2^4+1*2^3+1*2^2+1*2^0=128 + 16 + 8 + 4 + 1 = 157$



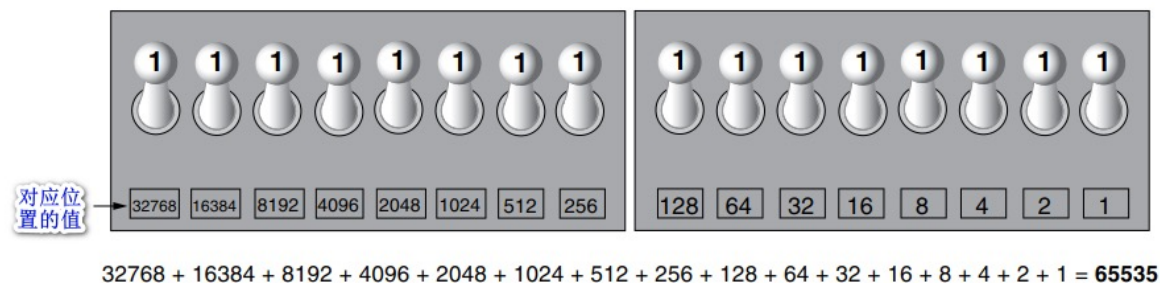
- 保存在存储单元中的值(value)可以是：

无符号整数

有符号整数

浮点数 (有理数)

字符(character)



这些值通过某种编码方式转换为二进制形式，并保存在存储单元中

比如：英文字符A保存在存储单元中时存储的是其对应的**ASCII码**65，即二进制的01000001

例子：ASCII码

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(nul)	0	0000	0x00	(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
(soh)	1	0001	0x01	!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
(stx)	2	0002	0x02	"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
(etx)	3	0003	0x03	#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
(eot)	4	0004	0x04	\$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
(enq)	5	0005	0x05	%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
(ack)	6	0006	0x06	&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
(bel)	7	0007	0x07	'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(bs)	8	0010	0x08	(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
(ht)	9	0011	0x09	)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
(nl)	10	0012	0x0a	*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
(vt)	11	0013	0x0b	+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
(np)	12	0014	0x0c	,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
(cr)	13	0015	0x0d	-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
(so)	14	0016	0x0e	.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
(si)	15	0017	0x0f	/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
(dle)	16	0020	0x10	0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
(dc1)	17	0021	0x11	1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
(dc2)	18	0022	0x12	2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
(dc3)	19	0023	0x13	3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
(dc4)	20	0024	0x14	4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
(nak)	21	0025	0x15	5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
(syn)	22	0026	0x16	6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
(etb)	23	0027	0x17	7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
(can)	24	0030	0x18	8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
(em)	25	0031	0x19	9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
(sub)	26	0032	0x1a	:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
(esc)	27	0033	0x1b	;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
(fs)	28	0034	0x1c	<	60	0074	0x3c	\	92	0134	0x5c		124	0174	0x7c
(gs)	29	0035	0x1d	=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
(rs)	30	0036	0x1e	>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
(us)	31	0037	0x1f	?	63	0077	0x3f	_	95	0137	0x5f	(del)	127	0177	0x7f

计算机编程概述

- 程序(program)：程序是用于控制计算机的一系列指令或代码(code)
- 程序语言：描述这些指令的语言，是“程序员”与“机器”对话的语言

语法(syntax)：哪些符号或文字的组合方式是正确的

语义(semantics)：描述程序的意义，当代码运行时计算机干什么

- 算法(algorithm)

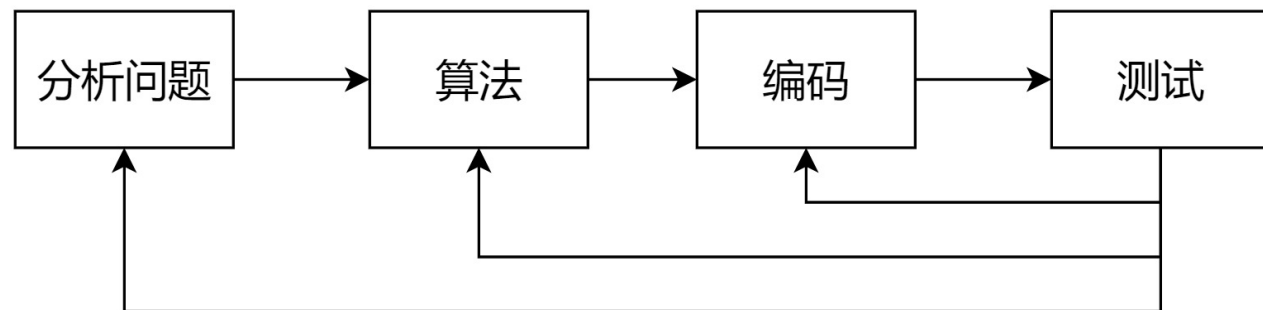
描述为了解决某个问题需要采取的动作以及这些动作的顺序

可以用自然语言、伪代码(pseudo-code, 自然语言混杂着一些程序代码)、程序流程图的形式描述

- 设计算法，然后将其转换为程序

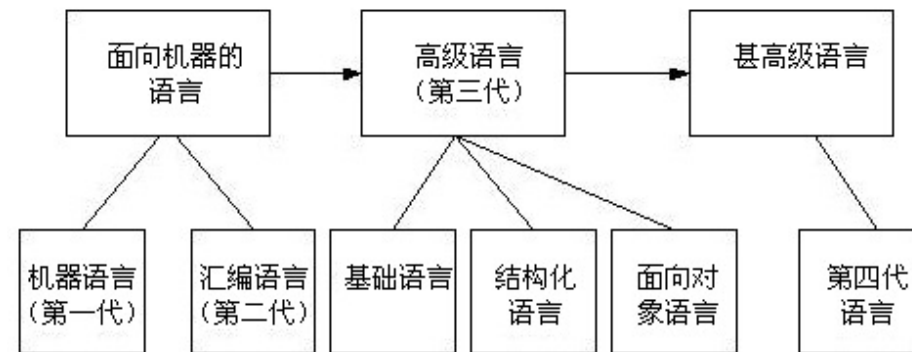


程序语言



计算机编程概述

• 程序(设计)语言的发展



前辈



低级语言

例如：0101110，汇编



高级码农



高级语言

例如：C、C++、Java、Python



任何人



智能语言

例如：领域语言、自然语言



机器人：Atlas

低级语言

- 机器语言 (Machine code) :

描述了CPU所执行的指令，由多个数字组成，比如 05 01 23

CPU从外存中将程序中的指令读取到内存中，然后根据指令进行相应的动作

- 汇编语言 (Assembly Language) : 以人可读的方式描述机器代码

```
add EAX, 1  
mov [ESP+4], EAX
```

- 不同架构的CPU有不同的指令集，如Intel、PowerPC等

- 经常需要大量的指令才能完成某个功能
- 难学、容易出错

程序(汇编语言)

```
add EAX, 1  
mov [ESP+4], EAX  
...
```

汇编器

Assembler

Intel (32和64位)

程序(机器语言)

```
10100001  
10110010  
...
```

M、

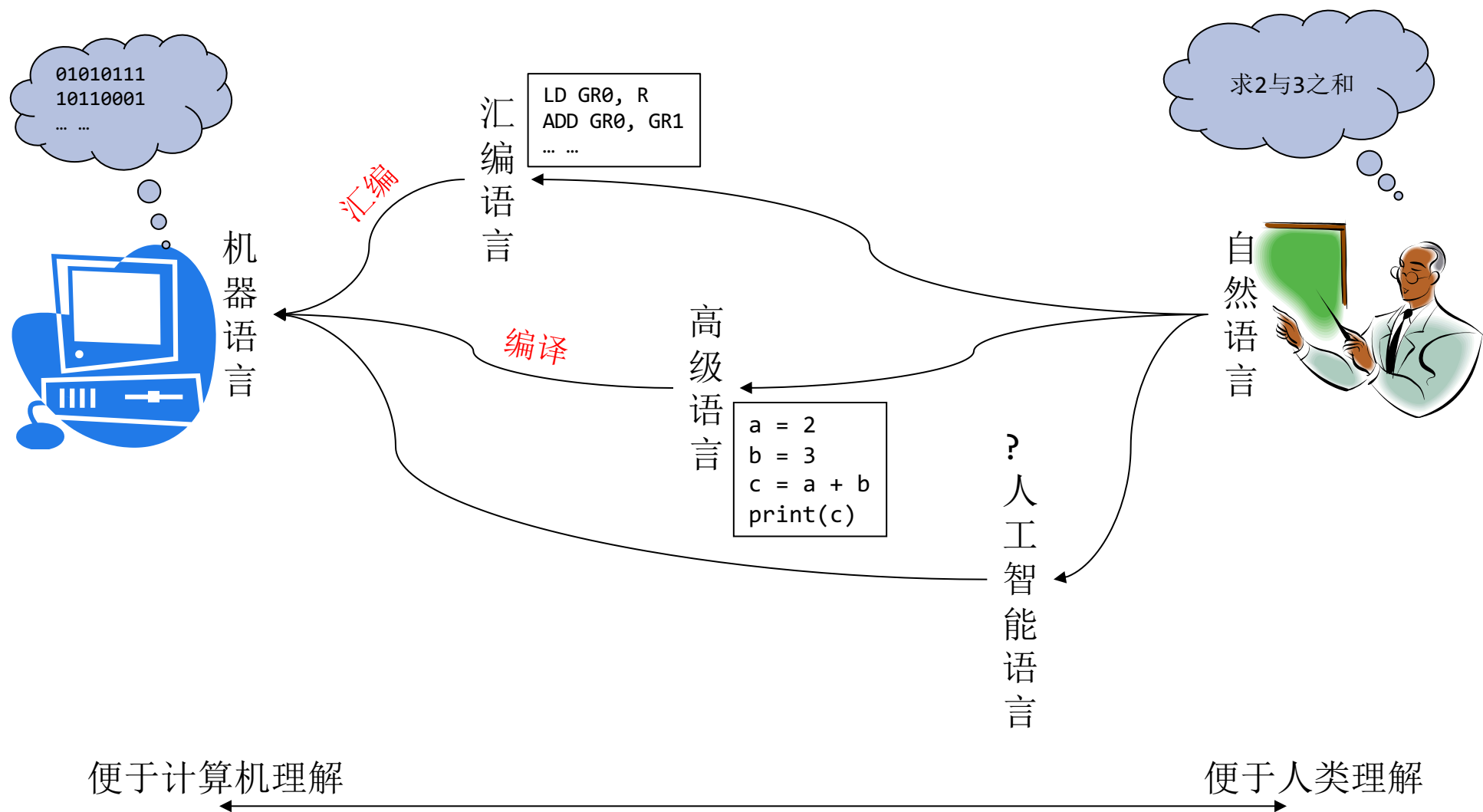
高级语言

- 类自然语言，不再依赖某种特定的机器或环境。在不同的平台上会被转换成不同的机器语言
- **语句(statement)**：高级语言中的一条**指令**，由关键字、运算符、标点符号以及其他编程元素（字面量等）按照相应的顺序组成
- 按照高级语言编写的程序称为**源程序**或源代码（**source code**）
- 更加容易学习和使用，一个语句经常对应着许多条机器代码
- Fortran, **C**, C++, C#, Java, Javascript, **Python**, **Visual Basic**, Ruby, Go等

```
total = price * (tax + 100, , 100
```

```
mov EAX, EBP[-2]
mov EBX, EBP[-4]
add EBX, 100
mul EAX, EBX
div EAX, 100
mov EBP[2], EAX
```

程序设计语言的种类



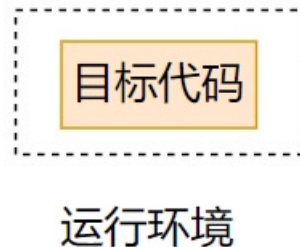
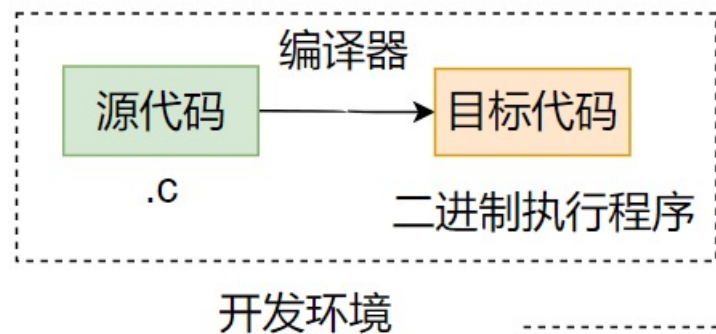
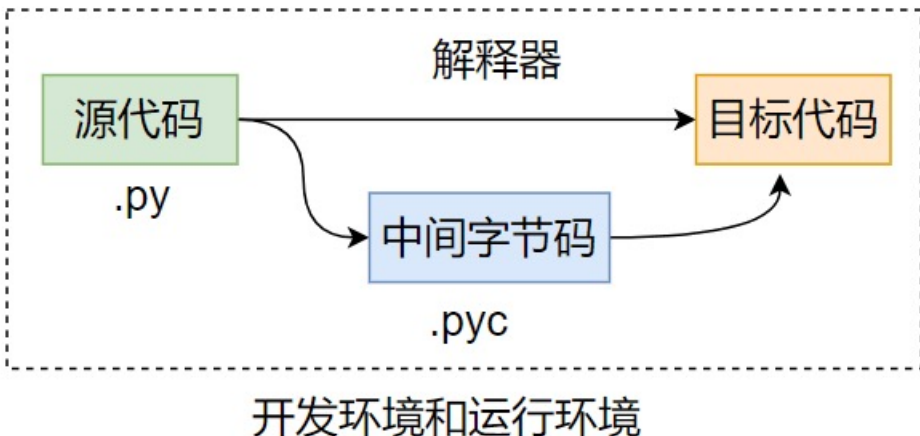
解释和编译

- 如何将使用高级语言编写的程序（源代码/源程序source code）转换为目标代码（机器码）？
- 解释器（Interpreter）：将语句翻译成机器码，并且执行，Python、Javascript、Perl、PHP等
 - 修改程序方便，修改代码重新运行就可以了
 - 必须有解释器才可以运行，跨平台（只要有解释器，就可以执行）
 - 每次运行，都要进行翻译，运行速度会有影响
- 编译器（Compiler）：将语句翻译成机器码，形成目标代码文件（机器语言），C、C++、VB等
 - 修改程序后需要进行编译
 - 编译一次，编译后的目标代码可以直接在相应的操作系统中运行，不再需要编译器
 - 编译时相比解释可以作更多的优化

- 有些语言将解释和编译结合在一起

Java语言：首先通过编译器(javac)将源代码转换为中间的Java字节码（Byte Code），然后在目标机器上通过解释器（java虚拟机）来运行

Python也支持伪编译。对于被import的模块源程序，首先将对应.py程序转换为.pyc字节码，优化程序和提高执行速度



大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串 更多的输出选项

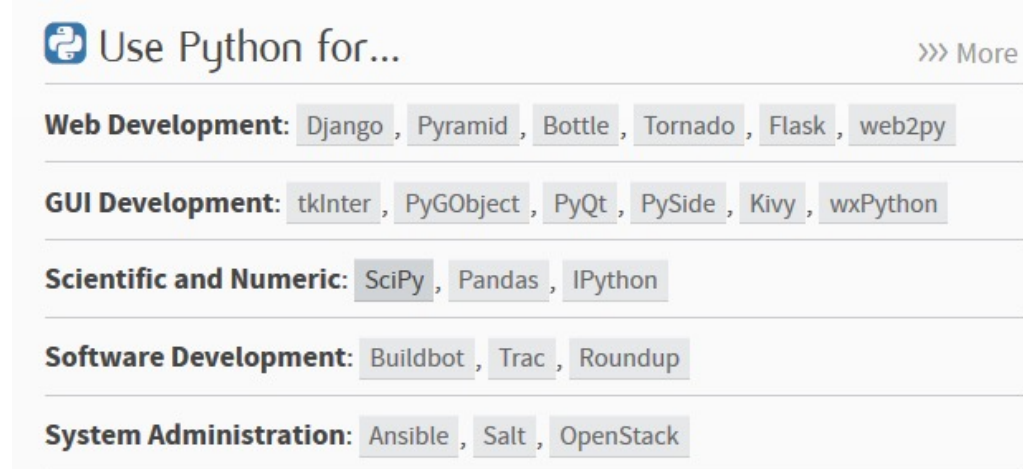
1.1 Python语言概述



- 1989年吉多·范罗苏姆Guido van Rossum发明Python [``paiθən`]语言
- Python以接近自然语言的风格诠释程序设计，最受欢迎的编程语言之一
- Python是一种**解释型**高级动态编程语言，通过丰富的**扩展库**支持，可广泛应用在Web应用、图形用户界面开发、系统管理、科学计算、人工智能、大数据、游戏等
- 可使用相关工具(比如<http://www.pyinstaller.org>)，将Python程序、解释器和第三方库打包转换为独立的可执行程序，可以在没有安装Python解释器和相关依赖包的平台上运行



终身仁慈独裁者
(BDFL, Benevolent Dictator
For Life)



Python版本

- Python2 :
2000年10月发布，2.7在2010年发布，最新版本为2020年4月发布的2.7.18
2020年1月已经停止python2的支持，不再继续开发（不再有2.8），仅仅修复bug

- Python3 : 我们的教学采用Python3
最初于2008年12月发布。最新版本为2021年2月发布的3.9.2
3.x版本与2.x版本并不完全兼容

- 可同时安装多个版本，通过菜单选择不同的Python版本

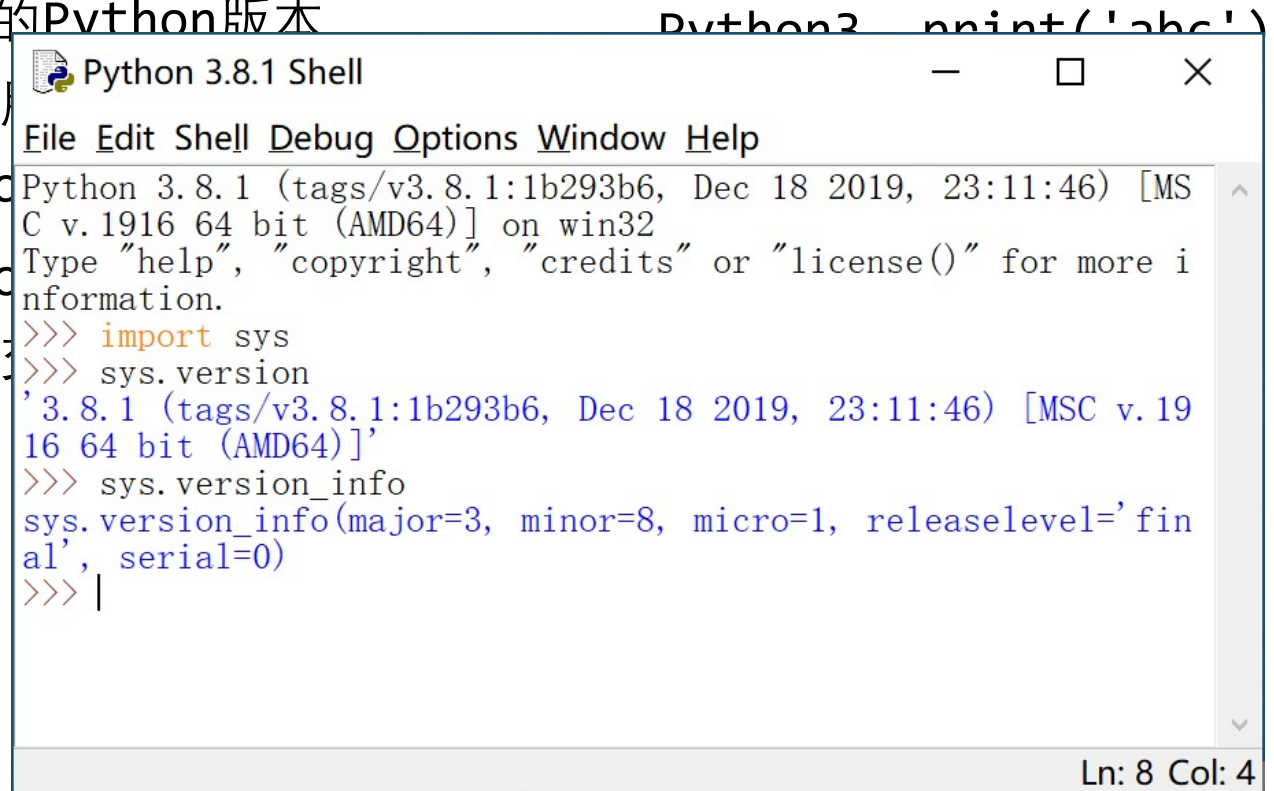
- 在所启动的IDLE界面可以直接看到对应的Python版本

- 绝大部分扩展库同时支持Python2和Python3

Python提供2to3的转换工具，将支持Python2的代码转换为Python3

- 查看已安装版本的方法（在所启动的IDLE3 Shell中）

```
import sys
sys.version
sys.version_info
```

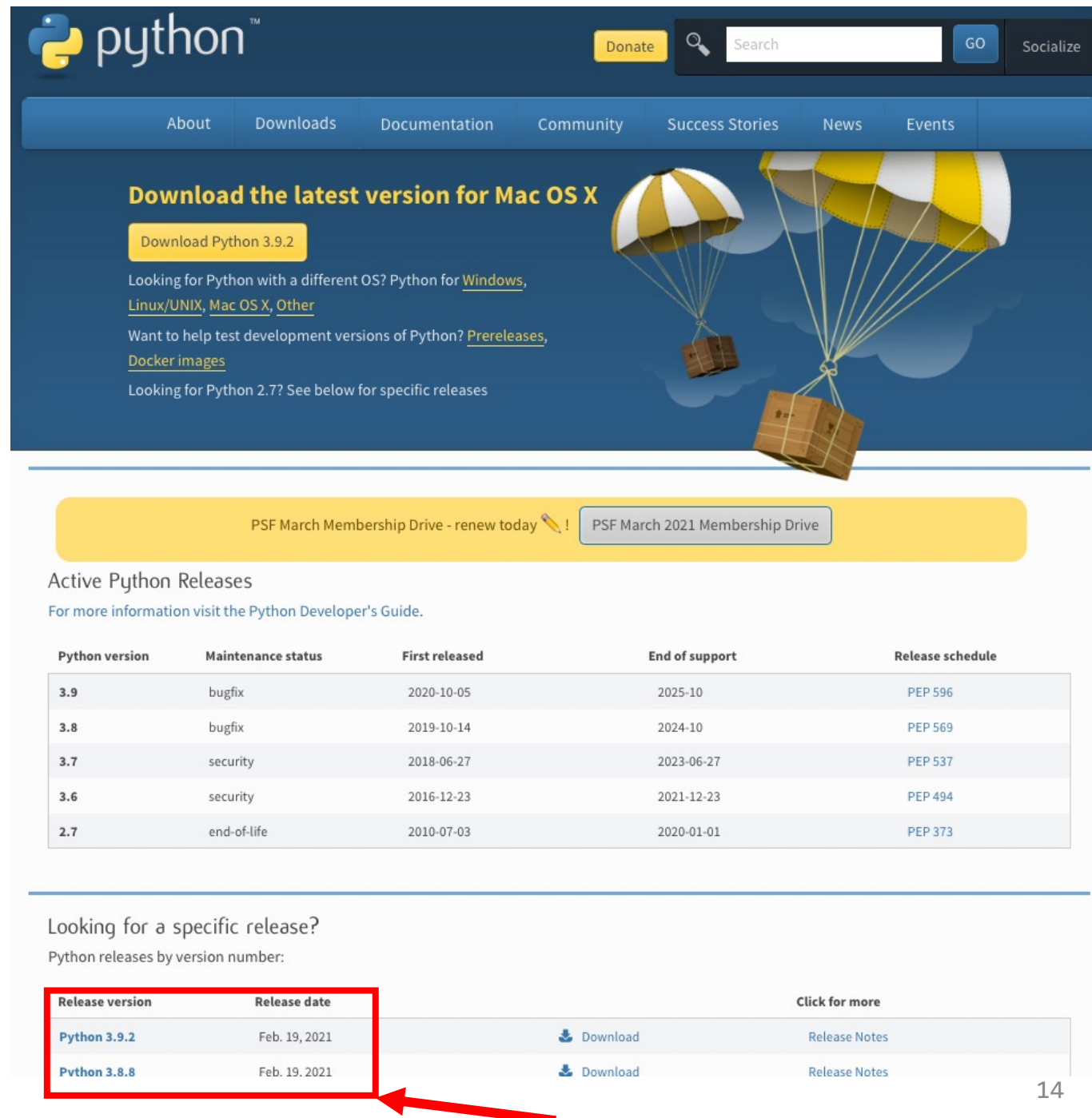
A screenshot of a Python 3.8.1 Shell window. The window title is "Python 3.8.1 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The text area shows the following output:

```
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import sys
>>> sys.version
'3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)]'
>>> sys.version_info
sys.version_info(major=3, minor=8, micro=1, releaselevel='final', serial=0)
>>> |
```

The status bar at the bottom right indicates "Ln: 8 Col: 4".

Python官方网站

- <https://www.python.org>



The screenshot shows the Python.org homepage. At the top is the Python logo and a navigation bar with links: About, Downloads, Documentation, Community, Success Stories, News, Events. A search bar and 'Donate' button are also present. The main banner features the text 'Download the latest version for Mac OS X' with a 'Download Python 3.9.2' button. Below this, links for other OSes (Windows, Linux/UNIX, Mac OS X, Other), pre-releases, Docker images, and Python 2.7 are provided. A yellow banner promotes the 'PSF March Membership Drive'. The 'Active Python Releases' section includes a table with columns: Python version, Maintenance status, First released, End of support, and Release schedule. The table lists versions 3.9, 3.8, 3.7, 3.6, and 2.7. Below this, a section titled 'Looking for a specific release?' shows 'Python releases by version number' with a table of recent releases. The first two rows of this table are highlighted with a red box and a red arrow points to them.

Download the latest version for Mac OS X

[Download Python 3.9.2](#)

Looking for Python with a different OS? Python for [Windows](#), [Linux/UNIX](#), [Mac OS X](#), [Other](#)

Want to help test development versions of Python? [Prereleases](#), [Docker images](#)

Looking for Python 2.7? See below for specific releases

PSF March Membership Drive - renew today ! PSF March 2021 Membership Drive

Active Python Releases

For more information visit the [Python Developer's Guide](#).

Python version	Maintenance status	First released	End of support	Release schedule
3.9	bugfix	2020-10-05	2025-10	PEP 596
3.8	bugfix	2019-10-14	2024-10	PEP 569
3.7	security	2018-06-27	2023-06-27	PEP 537
3.6	security	2016-12-23	2021-12-23	PEP 494
2.7	end-of-life	2010-07-03	2020-01-01	PEP 373

Looking for a specific release?

Python releases by version number:

Release version	Release date	Click for more
Python 3.9.2	Feb. 19, 2021	Download Release Notes
Python 3.8.8	Feb. 19, 2021	Download Release Notes

1.2 Python安装

- <https://www.python.org/downloads/> 根据所使用的平台 (Linux/Mac OS X/Windows) 下载相应的版本 (最新版本为3.9.2)

Version	Operating System	Description
Gzipped source tarball	Source release	
XZ compressed source tarball	Source release	
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later
Windows help file	Windows	
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64
Windows x86 embeddable zip file	Windows	
Windows x86 executable installer	Windows	
Windows x86 web-based installer	Windows	

- 可根据需要下载64位或者32位版本
 - X86-64: 表示64位版本
 - X86: 表示32位版本
- 红圈标出的是windows系统的64位和32位版本
- 紫圈标出的为Mac系统的64位版本
- 其他版本还包括
 - Web-based: 需要通过联网完成安装
 - Embeddable Zip File: 嵌入式版本, 将Python集成到其他应用中使用, 普通用户一般不会使用该版本

Python安装

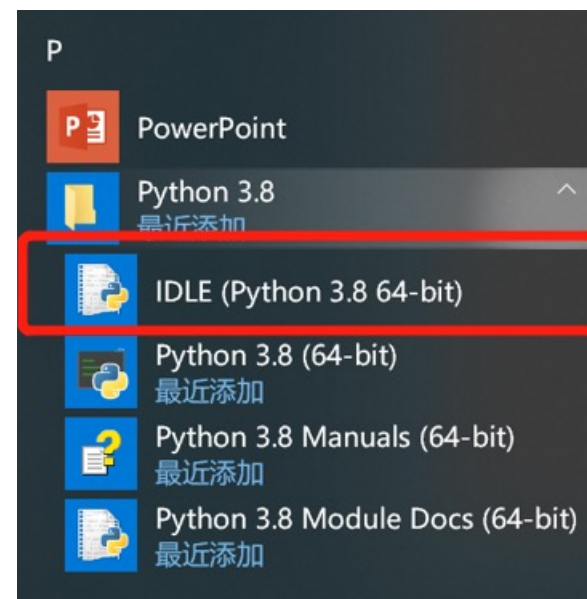
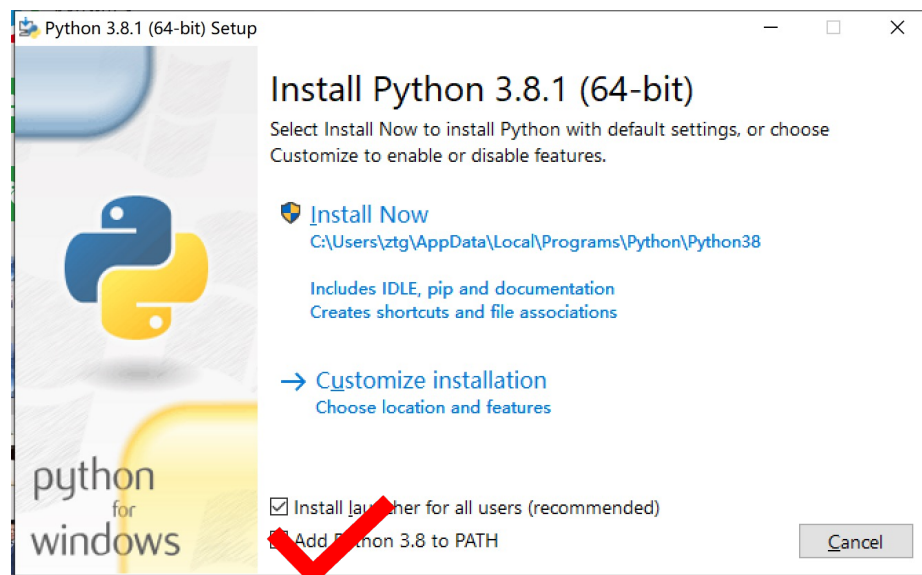
- Python解释器：

<https://www.python.org/downloads/> 根据所使用的平台（Linux/Mac OS X/Windows）下载相应的版本

Install Now: 按照缺省设置安装

Customize installation: 可以修改安装目录等设置

Install Launcher for all users

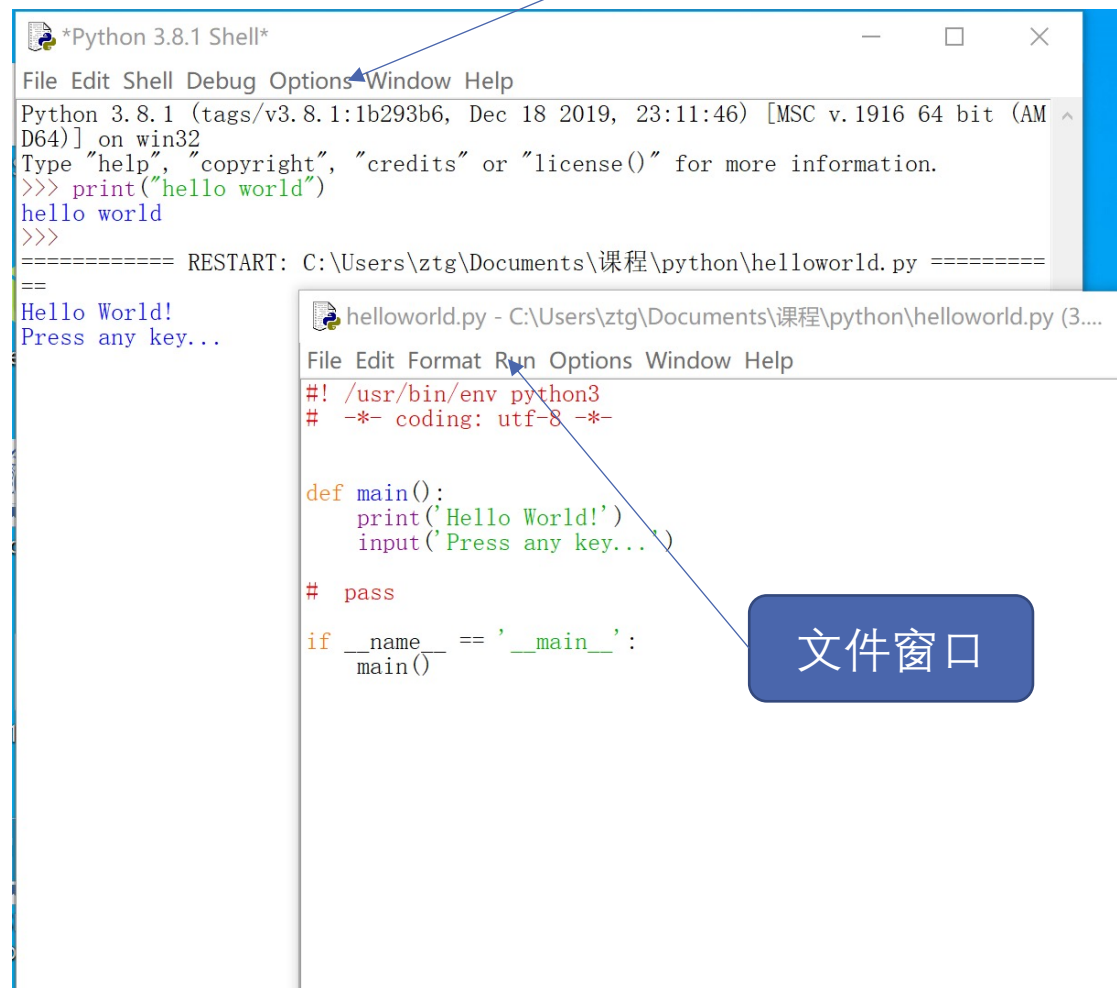


Python使用: **IDLE** 集成开发环境IDE

(Integrated Development Environment)

交互式窗口

- 将编辑器、调试器、解释器环境综合在一起
- 下载安装的Python环境中包括了**IDLE**
- **File**→New File: **.py**
- **Run**→Run Module来运行程序
- 其他常用的IDE还包括PyCharm、VS Code, Spyder、Jupyter等
- 通过集成开发环境编写程序是我们最常用的方式
- 期末考试采用**IDLE**开发环境



文件窗口

交互式窗口



The screenshot shows a Python 3.8.1 Shell window with a menu bar (File, Edit, Shell, Debug, Options, Window, Help) and a title bar (Python 3.8.1 Shell). The window content displays the following text:

```
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello world!")
Hello world!
>>>
===== RESTART: C:\Users\ztg\Documents\课程\python\helloworld.py =====
>>>
Hello World!
Press any key...
>>> 3+2
5
>>> 3*4
12
>>> 4**2
16
>>> |
```

shell窗



```
>>> 3.14*15*15
706.5
>>> (5+5)*2
20
>>> print("Hello world!")
Hello world!
>>> text = input("请输入")
请输入python
>>> print("您输入的是",
text)
您输入的是 python
```

Python解释器的提示符为 `>>>`

Read-Evaluate-Print Loop

(REPL) : **交互式模式** ,

解释器输出提示符 `>>>`

等待用户输入python语句

解释器执行该语句

如果是表达式则输出表达式的值，然后输出提示符，如此继续....

可以当成计算器来使用

也可以输出某些文本(print)

也可以等待用户输入(input)

基本编程套路：四部曲

- 一想 (Thinking)
- 二画 (Designing)
- 三码 (Coding)
- 四测 (Testing)

Python之禅 (The Zen of Python)

import

```
Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>>
```

Ln: 7 Col: 33

—— Tim Peters

优美胜于丑陋。

显式胜于隐式。

简单胜于复杂。

复杂胜于难懂。

扁平胜于嵌套。

分散胜于密集。

可读性应当被重视。

尽管实用性会打败纯粹性，特例也不能凌驾于规则之上。

除非明确地使其沉默，错误永远不应该默默地溜走。

面对不明确的定义，拒绝猜测的诱惑。

用一种方法，最好只有一种方法来做一件事。

虽然一开始这种方法并不是显而易见的，但谁叫你不是

Python之父呢。

做比不做好，但立马去做有时还不如不做。

如果实现很难说明，那它是个坏想法。

如果实现容易解释，那它有可能是个好想法。

命名空间是个绝妙的想法，让我们多多地使用它们吧！

优雅、明确、简单

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python基础知识：字符串
- Python代码编写规范
- Python编程快速入门

1.4.8 基本输入输出:简单输出

用print()在括号中加上字符串，就可以向屏幕上输出指定的文字。如下：

```
>>> print('hello, world')
hello, world
>>> print('The quick brown fox', 'jumps over', 'the lazy dog')
The quick brown fox jumps over the lazy dog
>>> print(300)
300
>>> print(100 + 200)
300
>>> print('100 + 200 =', 100 + 200)
100 + 200 = 300
```

print()函数也可以接受多个字符串，每个字符串之间用逗号分隔，字符串之间默认会加上一个空格，也可以不用逗号，而用加号连接。

遇到换行符\n，print()函数会自动换行。

也可用逗号分隔，输出后默认用空格分隔开来。

间用逗号分隔，输出后默认用
空格分隔开来

1.4.8 基本输入输出:简单输入

- Python提供了一个`input(prompt)`，首先输出`prompt`，等待用户输入，直到用户按回车结束，返回用户输入的字符串（不包括最后的回车）

```
>>> name = input()
```

```
Michael
```

```
>>> name
```

```
'Michael'
```

```
>>> name = input()
```

```
Michael
```

```
>>> name
```

```
'Michael'
```

- `input()`可以让你显示一个字符串来提示用户输入信息内容

```
name = input('please enter your name')
```

```
print('hello,', name)
```

```
>>> name = input('please enter your name: ')
```

```
please enter your name: 肖晓春
```

```
>>> name
```

```
'肖晓春'
```

```
>>> print("hello,", name)
```

```
hello, 肖晓春
```

在Idle交互式窗口练习

- 在交互式窗口演示

- 练习

2+3

```
print('hello, world')
```

```
print('The quick brown fox', 'jumps over', 'the lazy dog')
```

```
print(300)
```

```
print(100 + 200)
```

```
print('100 + 200 =', 100 + 200)
```

```
name = input()
```

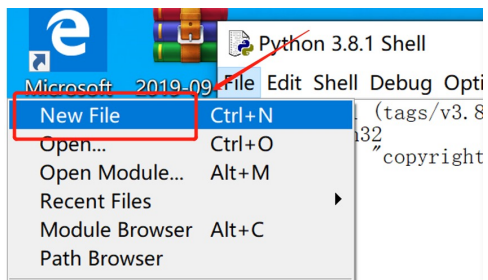
```
name
```

```
name = input('please enter your name: ')
```

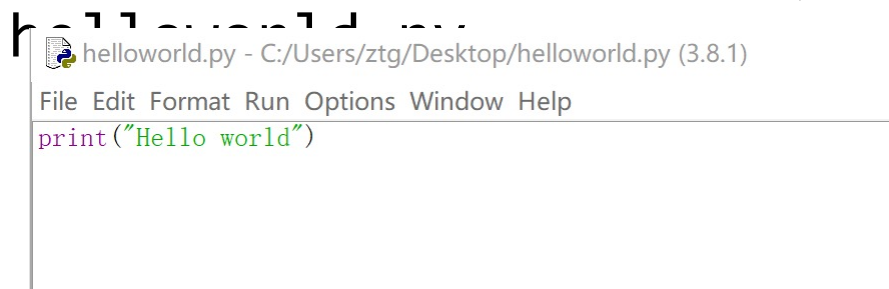
```
print('hello,', name)
```

你的第一个程序文件

- 由 交互式窗口 **File >> New File** 启动空白的文件窗口



- 文件窗口：如下编辑，保存(文件窗口 **File >> Save**) 文件



- 文件窗口：Run子菜单

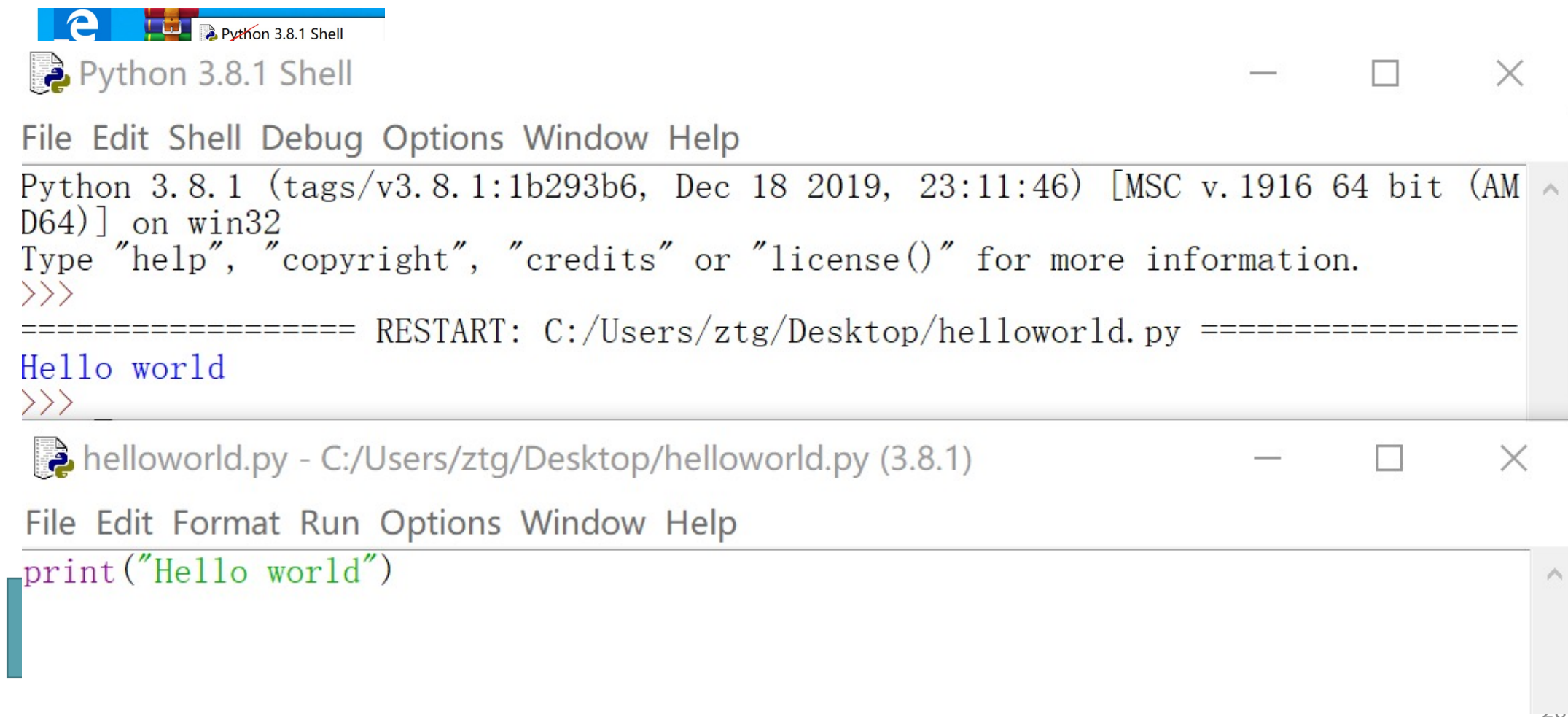
Run→Python Shell: 切换到交互Shell窗口

Run→Run Module(F5)运行，可以在Python Shell窗口中看到运行结果

Run→Check Module仅仅检查编写的程序是否有语法错误，并不运行

你的第一个程序文件

- 由 交互式窗口 **File >> New File** 启动空白的文件窗口



```
Python 3.8.1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/ztg/Desktop/helloworld.py =====
Hello world
>>>

helloworld.py - C:/Users/ztg/Desktop/helloworld.py (3.8.1)
File Edit Format Run Options Window Help
print("Hello world")
```

在Idle文件窗口练习

- 在Idle文件窗口中对helloworld.py文件进行修改，使得程序有下面两条语句组成，运行结果如图所示：

```
name = input('please enter your name: ')\nprint('hello,', name)
```

IDLE快捷键

option→config IDLE→keys → force-open-completions

- 撤销 (Ctrl+Z)、全选 (Ctrl+A)、复制 (Ctrl+C)、粘贴 (Ctrl+V)、剪切 (Ctrl+X)

快捷键	功能说明
Alt+p	浏览历史命令（上一条）
Alt+n	浏览历史命令（下一条）
Ctrl+F6	重启Shell，之前定义的对象和导入的模块全部失效
F1	打开Python帮助文档
Alt+/ 	（Expand word）自动补全前面曾经出现过的单词，如果之前有多个单词具有相同前缀，则在多个单词中循环选择
Tab	智能缩进或者自动完成（即输入前缀后会列出相关的关键字和属性）
CTRL+Space建议改成ALT+ \	(show completions)显示自动完成
CTRL + \	(show calltip)显示函数的参数使用方式
Ctrl+]	缩进代码块
Ctrl+[	取消代码块缩进
Alt+3	注释代码块
Alt+4	取消代码块注释。

Python其他的启动方式:

1.菜单选择Python控制台

- 通过菜单选择Python控制台(Console)

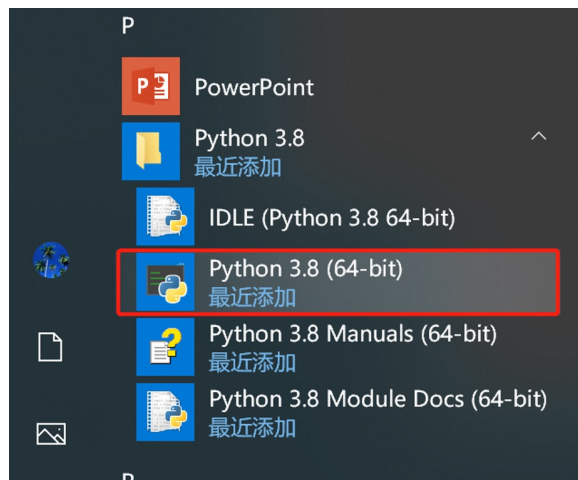
Python解释器的提示符为 >>>

Read-Evaluate-Print Loop (REPL) : 输入语句, 解释执行并输出结果

表达式的值如果不为None, 会输出其值, 可充当计算器的功能

解释器环境中_表示上一次运算的结果

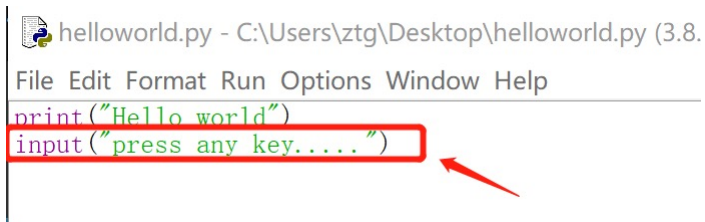
quit()退出

A screenshot of the Python 3.8 (64-bit) console window. The window title is 'Python 3.8 (64-bit)'. The console output shows the following interaction:

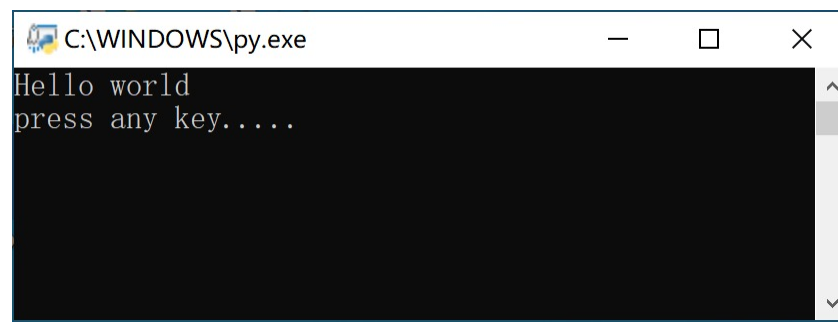
```
Python 3.8.1 (tags/v3.8.1:1b293b6, Dec 18 2019, 23:11:46) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+2
5
>>> print("hello, world")
hello, world
>>> name = input("Your name:")
Your name:Mike
>>> name
'Mike'
>>> print("Your name is ", name)
Your name is  Mike
>>> _
```

2.双击Python程序

- 资源管理器中双击扩展名为 “.py” 或 “.pyc” 的Python程序文件直接运行，**但是不建议采用这种方式！！！！**
- 操作系统会调用合适的Python解释器执行代码，**如果代码全部执行完，则关闭窗口退出**（程序有错时，出现异常，导致程序结束运行，然后窗口马上被关闭）
- 练习：
- 双击你刚才编写的helloworld.py（发现运行结果一闪而过）
- 为你的程序添加一句：`input(“Press any key...”)`，保存双击运行



```
helloworld.py - C:\Users\ztg\Desktop\helloworld.py (3.8.  
File Edit Format Run Options Window Help  
print("Hello world")  
input("press any key.....")
```



```
C:\WINDOWS\py.exe  
Hello world  
press any key.....
```

3. Python使用:命令行

- **命令提示符**环境运行Python程序文件

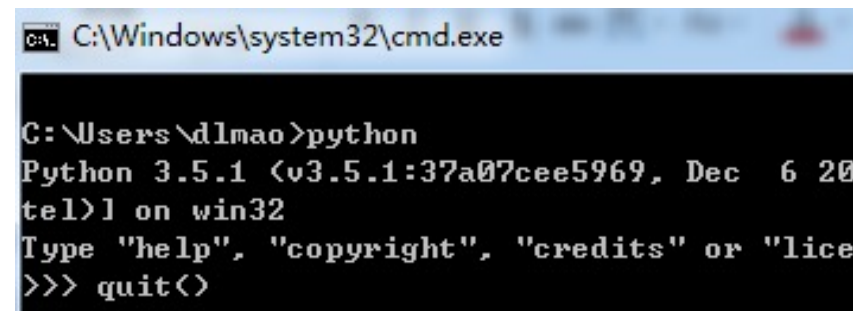
windows系统：

在“开始”菜单的“附件”中单击“**命令提示符**”

点击左下角任务栏上的"windows图标"，键入cmd搜索得到**命令提示符**

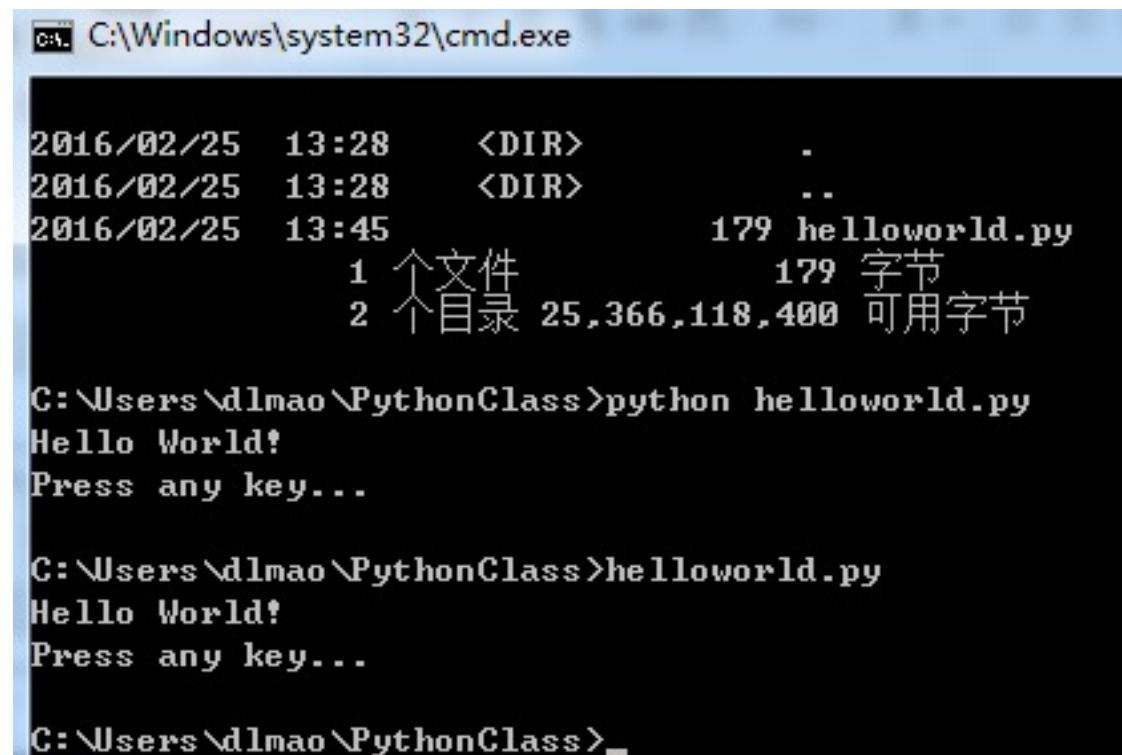
Linux或MacOS进入**终端应用**

- python进入python交互式控制台
- python helloworld.py调用解释器执行该程序后退出
- helloworld.py 执行python脚本



```
C:\Windows\system32\cmd.exe

C:\Users\dlmao>python
Python 3.5.1 <v3.5.1:37a07cee5969, Dec  6 2016>
Type "help", "copyright", "credits" or "license()" for more
>>> quit()
```



```
C:\Windows\system32\cmd.exe

2016/02/25  13:28    <DIR>          .
2016/02/25  13:28    <DIR>          ..
2016/02/25  13:45                179 helloworld.py
                1 个文件              179 字节
                2 个目录 25,366,118,400 可用字节

C:\Users\dlmao\PythonClass>python helloworld.py
Hello World!
Press any key...

C:\Users\dlmao\PythonClass>helloworld.py
Hello World!
Press any key...

C:\Users\dlmao\PythonClass>
```

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串
- Python编程快速入门

1.4 Python基础知识: 对象 (Object)

- 对象：Python中各种数据（信息）的抽象

标识ID(identity): 对象一旦创建其ID不再改变，可以看成该对象在内存中的地址，

id(x) 返回对象x的ID， a is b 判别 a和b是否同一个对象（ID是否相同）

类型(type)：决定了对象可能取值的范围以及支持的操作。对象的类型不可变，强类型语言

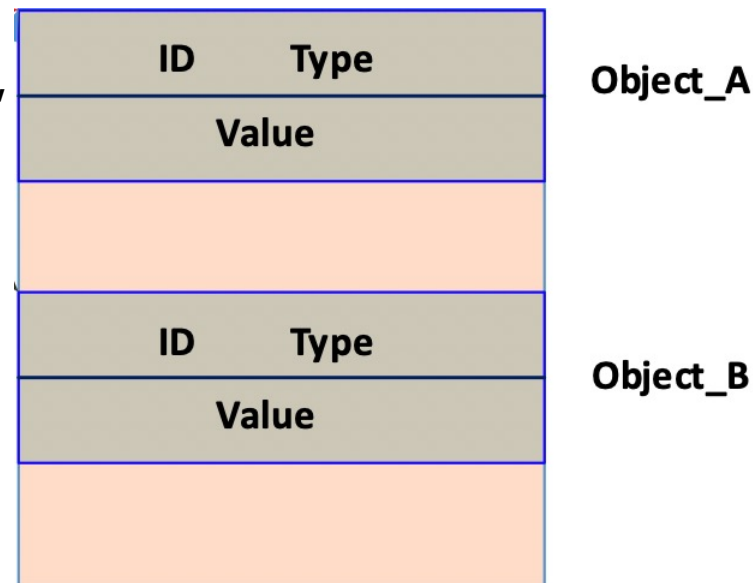
type(x) 返回对象x的类型

对象是类型的一个实例（instance）

值(value): 值可以被改变或者不可变，决定于其类型

不可变(immutable)：有些对象一旦创建其值不可变，比如数字、字符串、元组等

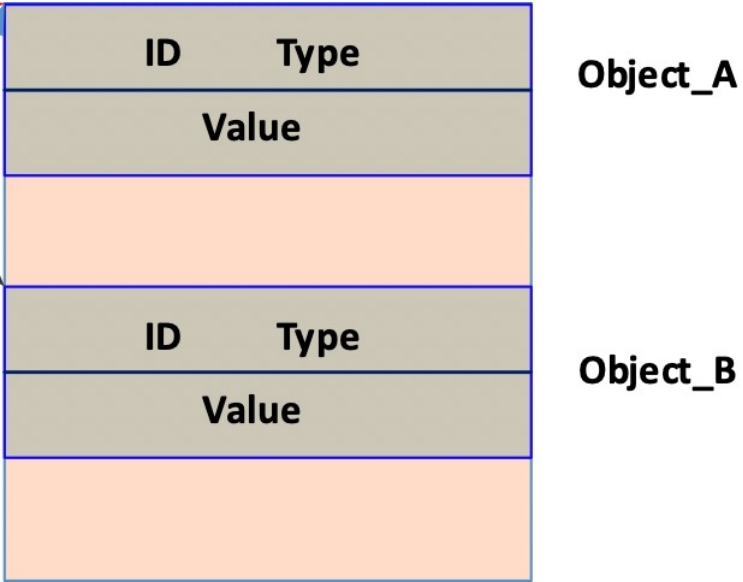
可变(mutable): 对象的值可以改变，比如列表、字典等



Python内置对象类型

对象类型	示例
数字	1234, 3.14, 3.14e-10 , 3+4j
字符串	'swfu', "I'm student", '''Python '''
列表	[1, 2, 3] ['a','b',['c',2]]
字典	{1:'food' ,2:'taste', 3:'import'}
元组	(2, -5, 6)
文件	f=open('data.dat', 'r')
集合	{'a', 'b', 'c'}
布尔型	True, False
空类型	None
编程单元类型	函数(def)、模块、类(class)

Literal(字面值)
表示某个内置对象类型的固定不变的值，在词法和语法分析时识别其类型



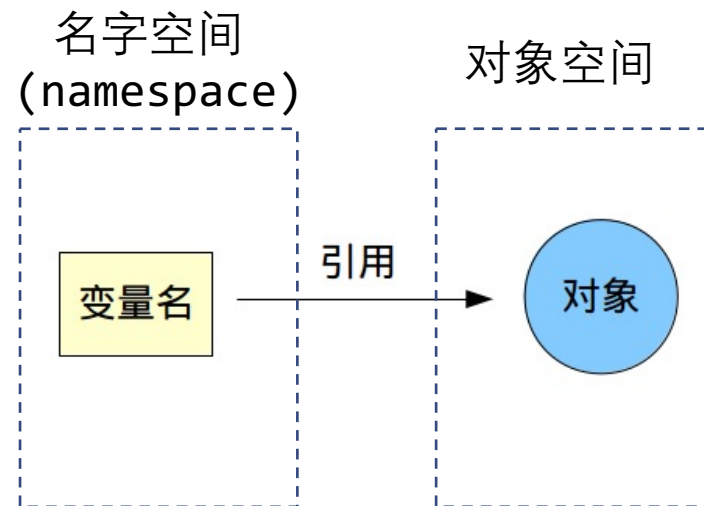
1.4 Python基础知识: 如何描述和引用对象?

- 可以通过**表达式(expression)**创建了**新的对象**:
如通过运算符(operator)运算之后的结果: $(2*4+1)*6$
- **变量(variable)**: 表示对于某个对象的引用 (reference)
在程序中通过变量名来描述
通过**赋值语句(assignment statement)**来给变量赋值:
variable = expression (LHS = RHS)

```
>>> a = (3*4+5)*6
变量102 >>> a
102 >>> |

有 >>> name = input()
Tom
>>> name
'Tom'
>>>
```

```
>>> print(x)
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    print(x)
NameError: name 'x' is not defined
>>> |
```



变量

- 由于变量表示对于某个对象的引用，因此Python允许多个变量指向同一个对象

```
>>> x = 3
```

```
>>> id(x)
```

```
1786684560
```

```
>>> y = x
```

```
>>> id(y)
```

```
1786684560
```

```
>>> x is y
```

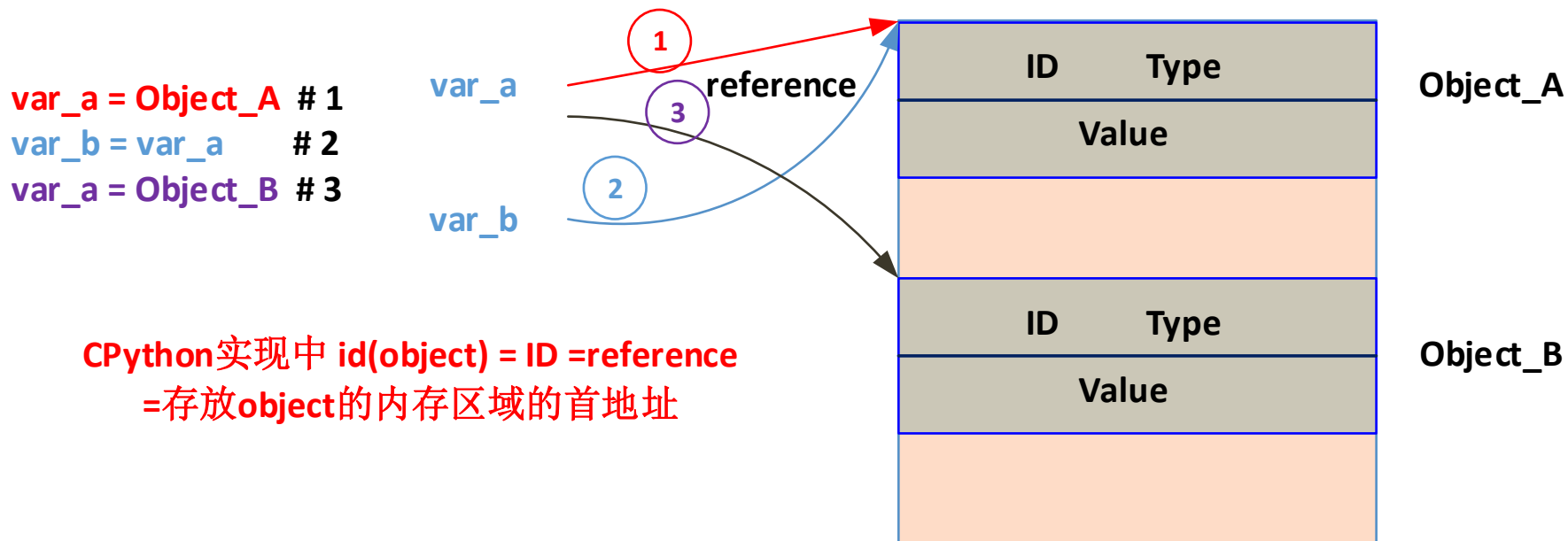
```
True
```

```
>>> type(x)
```

```
<class 'int'>
```

```
>>> isinstance(x,int)
```

```
True
```



- 内置函数 `id(object)` 用来返回object的ID
- 表达式 `a is b` 判别 a和b是否同一个对象 (ID是否相同)
- 内置函数 `type(object)` 用来返回object的类型
- 内置函数 `isinstance(obj, class)` 用来测试对象obj是否为指定类型class的实例

变量

* 变量出现在LHS，表示给该变量赋值，即保存RHS所对应对象的引用（地址）

如：`x = 3`

* 变量出现在RHS(Right Hand Side) 处时表示引用该变量所指向对象的值

如：`y = x`

- 一个变量赋予一个新的值后，变量指向新的对象

```
>>> x = x + 6
```

```
>>> id(x)
```

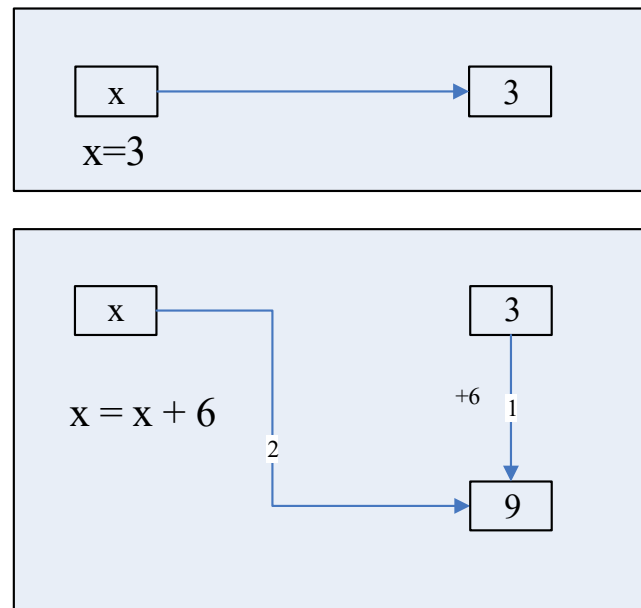
```
1786684752
```

```
>>> y
```

```
3
```

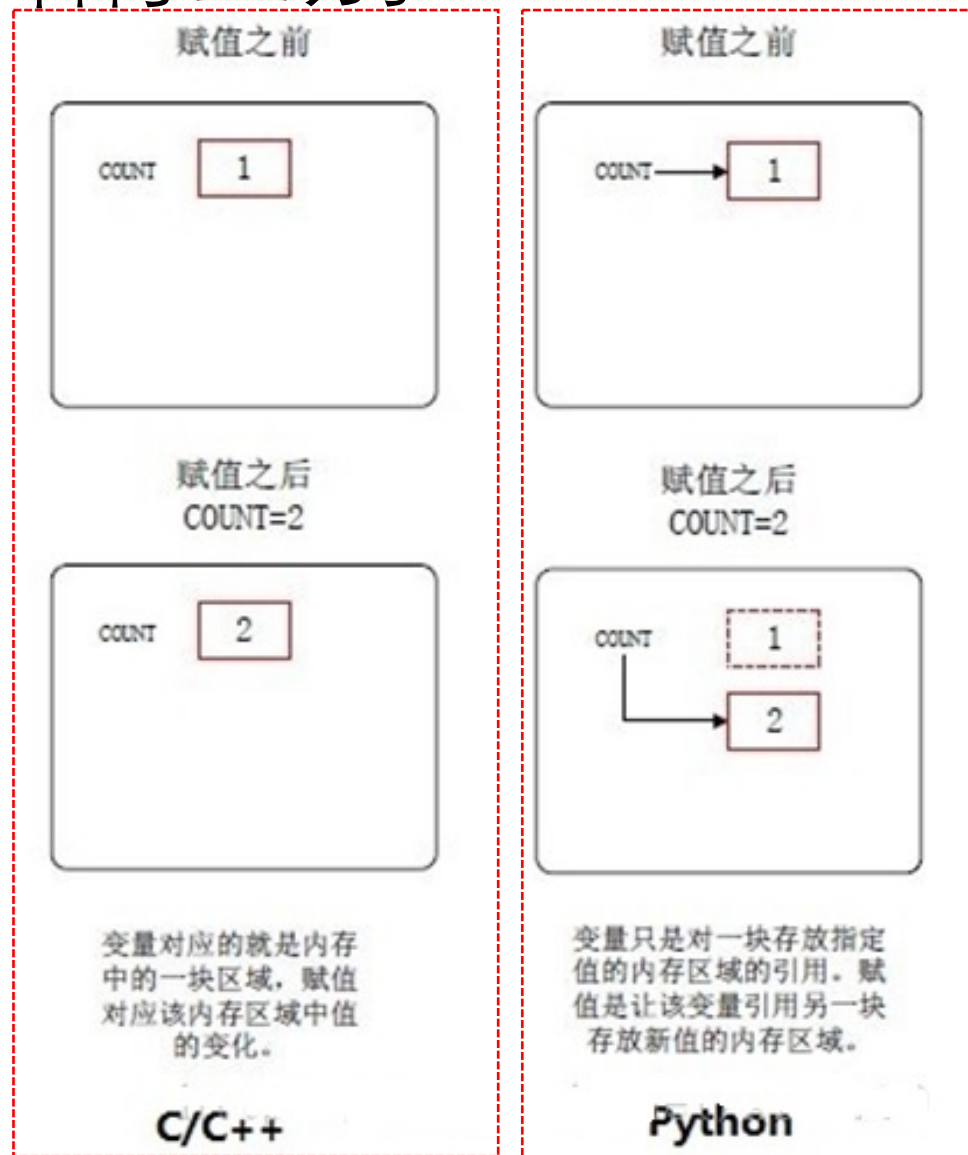
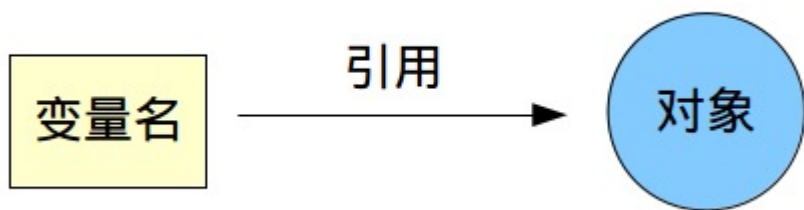
```
>>> id(y)
```

```
1786684560
```



python变量与其他语言的区别

- Python语言中变量仅仅是对象的引用
- C/C++中变量对应的是数据(对象)



变量：名字与对象的绑定

- Python是一种动态类型语言：变量的类型是可以随时变化的

```
>>> x = 3
```

```
>>> print(type(x))
```

```
<class 'int'>
```

```
>>> x = 'Hello world.'
```

```
>>> print(type(x))
```

```
<class 'str'>
```

```
>>> x = [1,2,3]
```

```
>>> print(type(x))
```

```
<class 'list'>
```

```
>>> isinstance(3, int)
```

```
True
```

```
>>> isinstance('Hello world', str)
```

```
True
```

在python语言中，类型是对象的一部分，
变量保存的是对象的引用

对象值:可变与不可变

- 不可变(`immutable`)对象:

有些对象一旦创建其值不可变, 比如数字、字符串、元组等

```
>>> x=(1,2,3)
>>> x[1]=4
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    x[1]=4
TypeError: 'tuple' object does not support item assignment
>>> x
(1, 2, 3)
>>>
```

- 可变(`mutable`):对象的值可以改变,

如列表、字典等

```
>>> x=[1,2,3]
>>> x[1]=4
>>> x
[1, 4, 3]
>>> |
```

复合赋值与链式赋值

- 复合赋值 (augmented assignment): `var op= expression` , `op` 为算术运算符,
 - 注意运算符和=之间没有空格, 相当于 `var = var op expression`
`sum += item`
`sum *= item`
`sum **= item`
 - 简化代码, 程序精炼
- 链式赋值 (chained assignment): `var1 = var2 = ... = expression`
 - 右结合, 即首先 `var2 = ....`; `var1 = var2`
`x = y = 123`

练习：

```
a = 3+4+6  
b = a + 7  
c = d +8
```

```
x = 3  
y = x  
x = x+6
```

```
a = (3*4+5)*6  
a = a*a
```

```
x = 3; y = x; x = x+6
```

Python一般一行一个语句，
但也可在同一行中放置多条语句，语句之间使用分号“;”
分割，但为可读起见，不建议。

对象值:相等==

- 比较运算符“==”
 - 判断两个对象的值是否相等，如果相等，运算结果为True，否则为False
- 与 比较运算符is 区别
 - 注意与赋值语句的区别：x = y 表示变量x与y所对应的对象绑定，即x也保存了y所对应对象的引用
 - 判断两个对象是否为同一个对象，如果是，结果为True，否则为False
 - 等价于 id(x) == id(y)

第一版教材P8最后一段的描述有误

“Python采用的是基于值的内存管理方式，如果为不同变量赋值为相同值，这个值在内存这两个只有一份，多个变量指向同一个内存地址”

```
>>> x = [1, 2]
>>> y = [1, 2]
>>> x is y
False
>>> x == y
True
>>> x = 1024
>>> y = 1024
>>> x == y
True
>>> x is y
False
```

```
>>> x = 3
>>> id(x)
10417624
>>> y = 3
>>> id(y)
10417624
>>> id(x) == id(y)
True
>>> x is y
True
```

实际上：

- 两个变量分别赋值为两个可变对象，值即便相同，也会有两个可变对象存在。
- 而对于不可变对象，由于其值不变，为了性能方面的考虑，有时Python实现中可能仅仅分配一个对象，多个变量指向同一个对象。

变量:不可变对象存放优化 (不作要求)

python的解释器实现考虑到小的整数出现的频率较高, 且为不可变对象, 重用相同的对象 (这样id相同)。对于字符串也有相应的优化

```
# int_optimization_test.py
print('[-10,269]的整数间,
for i in range(-10,270):
    a = int(str(i))
    b = int(str(i))
    # 仅仅打印那些值相同而
    if a is not b:
        print(i)
```

```
>>>
RESTART: /Users/xiaochunxiao/Documents/课程
ptimization_test.py
[-10,269]的整数间, 没有进行优化的整数如下:
-10
-9
-8
-7
-6
257
258
259
260
261
262
263
264
265
266
267
268
269
>>> |
```

- 将代码copy到文件窗口, 存成 `int_optimization_test.py` 文件, 运行看下运行结果

变量:不可变对象存放优化 (不作要求)

python的解释器实现考虑到小的整数出现的频率较高, 且为不可变对象, 重用相同的对象 (这样id相同)。对于字符串也有相应的优化

```
# int_optimization_test.py
print('[-10,269]的整数间,
for i in range(-10,270):
    a = int(str(i))
    b = int(str(i))
    # 仅仅打印那些值相同而
    if a is not b:
        print(i)
```

```
>>>
RESTART: /Users/xiaochunxiao/Documents/课程
ptimization_test.py
[-10,269]的整数间, 没有进行优化的整数如下:
-10
-9
-8
-7
-6
257
258
259
260
261
262
263
264
265
266
267
268
269
>>> |
```

- 将代码copy到文件窗口, 存成 `int_optimization_test.py` 文件, 运行看下运行结果
- 本例子中说明整数为不可变对象, 而且注意只有在 `[-5,256]` 间的整数才有这样的行为, 而且不排除以后实现可能不再适用。

1.4.7 对象的删除

- Python具有自动内存管理功能

解释器会跟踪所有的对象的引用情况，一旦发现某个对象不再有任何变量引用，垃圾回收机制在适当的时候会回收该对象，释放内存资源

del语句可显式解除变量与所指向对象之间的绑定，即删除了变量

del语句可解除列表list等可变序列中的元素的绑定，即删除该元素。

```
>>> x = [1,2,3,4,5,6]
>>> y = 3
>>> z = y
>>> print(y)
3
>>> del y #删除绑定（严格来讲，是删除了变量）
>>> print(y)
Traceback (most recent call last):
  File "<pyshell#52>", line 1, in <module>
    print(y)
NameError: name 'y' is not defined
>>> print(z)#3这个对象还在
3
```

```
>>> del z
>>> print(z)
Traceback (most recent call last):
  File "<pyshell#56>", line 1, in <module>
    print(z)
NameError: name 'z' is not defined
>>> del x[1] #删除列表中指定元素
>>> print(x)
[1, 3, 4, 5, 6]
>>> del x #删除整个列表（严格来讲，是删除了指向列表变量）
>>> print(x)
Traceback (most recent call last):
  File "<pyshell#60>", line 1, in <module>
    print(x)
NameError: name 'x' is not defined
```

变量：使用`help`和`dir`函数查看帮助信息

<code>dir()</code>	列出当前名字空间中的所有名字
<code>dir(object)</code>	列出 <code>object</code> 相关的属性和方法列表，其中下划线(<code>_</code>)开头的名字表示内部使用，如 <code>dir(str)</code> , <code>dir('hello')</code>
<code>help(object)</code>	查看对象 <code>object</code> 相关的帮助，比如 <code>help(print)</code> 查看函数 <code>print</code> 的相关帮助， <code>help(3.14)</code> 查看浮点数支持的属性和方法， <code>help(str)</code> 查看类型为 <code>str</code> 的字符串对象的属性和方法
<code>help()</code>	进入交互式的Help系统，键入 <code>quit</code> 退出帮助系统
<code>help('topic')</code>	查看交互式help系统的某个主题(<code>topic</code>) 相关的帮助，比如 <code>help('if')</code> 查看 <code>if</code> 语句语法

help

- **help('topic'):** 查看某个主题(topic) 相关的帮助

```
>>> help('if')
The "if" statement
*****
```

The "if" statement is used for conditional execution:

```
if_stmt ::= "if" expression ":" suite
          ( "elif" expression ":" suite ) *
          ["else" ":" suite]
```

It selects exactly one of the suites by evaluating the expressions one by one until one is found to be true (see section Boolean operations for the definition of true and false); then that suite is executed (and no other part of the "if" statement is executed or evaluated). If all expressions are false, the suite of the "else" clause, if present, is executed.

Related help topics: TRUTHVALUE

```
>>> help()

Welcome to Python 3.8's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at https://docs.python.org/3.8/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics". Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".

help> int
Squeezed text (255 lines).
help> quit

You are now leaving help and returning to the Python interpreter.
If you want to ask for help on a particular object directly from the
interpreter, you can type "help(object)". Executing "help('string')"
has the same effect as typing a particular string at the help> prompt.
>>>
```

- **help(thing):** 查看对象thing相关的帮助

```
>>> help(1)
Help on int object:
```

```
class int(object)
| int(x=0) -> integer
| int(x, base=10) -> integer
|
| Convert a number or string to an integer, or return 0 if no arguments
| are given. If x is a number, return x.__int__(). For floating point
| numbers, this truncates towards zero.
|
```

- **help():** 进入交互式的Help系统，键入quit退出帮助系统

```
>>> help
help> int
Help> quit
```

dir

- **dir():** 列出当前名字空间中的所有名字

```
>>> x = 1
>>> y = 3
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'x', 'y']
>>> |
```

- **dir([object]):** 查看object相关的属性和方法列表，其中_开头的名字表示内部

部

```
>>> dir(1)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__', '__getattribute__', '__getnewargs__', '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__', '__xor__', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'imag', 'numerator', 'real', 'to_bytes']
```

- `[n for n in dir([object]) if not n.startswith('_')]` 过滤掉以_开头的属性

变量名命名规则

- 标识符是变量、函数、类、模块和其他对象的名字
- 标识符必须以字母或下划线开头，其后的字符可以是字母、下划线或数字
以下划线开头的变量在Python中有特殊含义
变量名中不能有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等等）
`99var It'OK first-class`
- 标识符对英文字母的大小写敏感，例如student和Student是不同的变量。
- 建议使用有意义的名字，i,j,k仅仅用在较短的循环等结构，比如area、keys等
- 建议使用小写字母表示变量名
多个单词时以下划线隔开：lower_case_with_underscores
或者使用驼峰式：lowerCaseWithUnderscores

查看Python所有关键字

- 一些特殊的标识符保留为Python关键字，不能用作变量名

```
import keyword
print(keyword.kwlist)
keyword.iskeyword('if') #判断是否为关键字
```

```
>>>help()
help>keywords
Help>quit
```

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>> as = 3.5
SyntaxError: invalid syntax
>>>
```

查看内置模块、类型、函数

- ❑ 不建议使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义，可以通过`dir(__builtins__)`查看所有内置模块、类型和函数；

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '_', '__build_class__', '__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
>>>
```